INDIVIDUAL CONTRIBUTION REPORT | Member - 2

Student name	S.S.Savindi Nawarthana
Student ID | UOW Number	20241374 | w2181807
Group number	Group 05
Member Role	Student - 2
Module	3D Interactive Media
Submission date	Thursday 12th of November 2026 at 17:30:00
GitHub	TreasureIslandGroupProject/Group05_CW1_UnityProject_TreasureIsland at student2-mouse-ui · rashennimsara/TreasureIsland-GroupProject

1. Introduction and project overview

This Island Treasure Hunter is a group-developed 3D interactive LEGO-style adventure set on a tropical island. The game concept involves a minifigure exploring the environment, interacting with obstacles and objects, and progressing towards hidden treasure. The group divided the project into separate responsibilities covering the island environment, character, treasure-related features, and interaction systems. My assigned role was Student 2: Mouse Interaction and User Interface (UI). This report explains the components I developed, how I used Unity and C# to implement them, the problems encountered, and how I shared my work through GitHub. It also distinguishes features that I personally tested from those that still require integration with the shared island scene and the final LEGO character.

2. My role and responsibilities

My initial responsibility was to implement mouse-based interaction with LEGO objects and island props, create a 2D Start/Resume/Quit interface, and contribute to an animated LEGO minifigure controlled by a UI button. During group discussion, the required mouse behaviour was clarified: gameplay should also use a third-person camera, with mouse movement changing the viewing angle, the cursor hidden during play, and the character turning as it moves. I responded by preparing reusable player and camera scripts, while retaining the original object-dragging feature. I worked in a separate Student2 test scene so that I would not overwrite the environment developer's scene.

Assigned feature	My contribution / current status
Mouse-input LEGO brick	Implemented and tested drag movement; saved reusable prefab.
Start / Resume / Quit UI	Designed menu and wrote menu-control script; full button test to confirm.
Third-person player + camera	Created and pushed reusable WASD, turning, camera and cursor scripts; final-scene test pending.
Animated LEGO minifigure + UI	Pending teammate's character asset; not yet integrated or verified.
Group integration	Pushed contribution to shared GitHub repository; merge and island integration to confirm.

3. Development and implementation

3.1 Mouse-controlled LEGO brick and rock

I began in Unity 6 using a dedicated scene named Student2_MouseTest. I imported a LEGO brick model, adjusted its scale, created a compatible URP/Lit material, and added a collider so Unity could detect mouse interaction. My MouseInput.cs script uses OnMouseDrag() and the Mouse X / Mouse Y axes to calculate horizontal movement. The object is translated in world space, allowing the user to drag it across the test ground. I applied the same interaction script to a rock imported from the Low Poly Rock Pack, configured its collider, and saved both objects as prefabs for reuse. In Play mode, I confirmed that the brick and rock could be moved with the mouse. This was useful for practising the collider, scripting, and prefab concepts from Tutorials 02 and 03.

3.2 Third-person movement, camera and cursor control

Following teammate feedback, I extended the interaction work beyond object dragging. ThirdPersonPlayerController.cs uses a CharacterController and the Horizontal / Vertical input axes to implement WASD movement relative to the active camera. The movement direction is calculated using flattened camera-forward and camera-right vectors. Quaternion.LookRotation and Quaternion.Slerp turn the character smoothly towards the movement direction. The script also applies gravity and uses CharacterController.Move to handle movement.

ThirdPersonMouseCamera.cs provides an orbiting third-person view. It follows an assigned player Transform, reads Mouse X and Mouse Y while the cursor is locked, and updates yaw and pitch. Vertical rotation is clamped to avoid unrestricted camera flipping. The camera position is recalculated at a configurable distance and height from the character. These scripts were written as reusable components because the teammate's final Mecabricks character had not yet been committed to the shared repository. They were pushed for integration but were not yet tested against the final character, island terrain or camera setup at the time of writing.

3.3 User interface and interactive props

I created a Canvas-based menu with a title, subtitle, Start Game, Resume Game, Quit Game and Menu controls, using a forest-green and gold visual theme to match the island. I positioned the Menu button in the top-right corner and configured the Canvas Scaler for a 1920 × 1080 reference resolution.

GameMenuController.cs controls menu visibility and Time.timeScale. I updated it so Start and Resume lock and hide the cursor, while opening the menu releases and shows the cursor. Escape toggles the pause menu because a locked cursor cannot click an on-screen Menu button during normal gameplay. The Quit method uses Application.Quit(), which requires a standalone build for a meaningful test.

The draggable brick and rock remain separate interaction features. I recognise that they cannot be dragged normally while the mouse is locked for third-person camera control. The group must therefore decide whether to retain object dragging in a separate interaction mode or use the third-person controller as the primary mouse-input feature

5. Testing, challenges and solutions

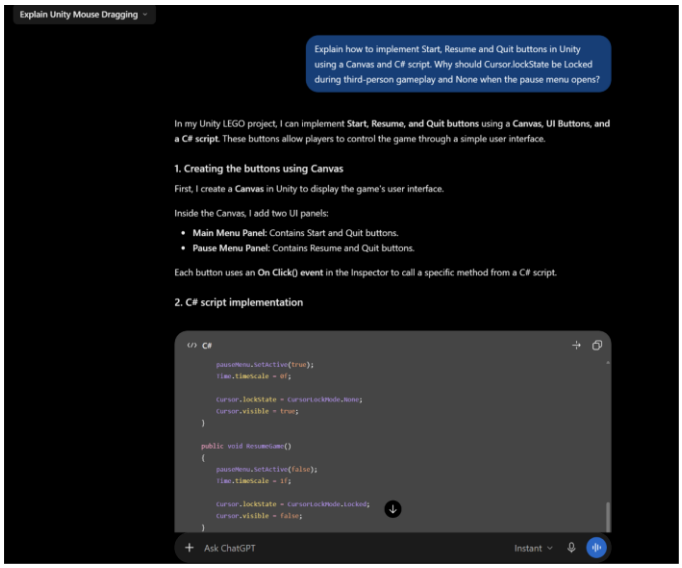
The main technical challenge was understanding that the original mouse-dragging tutorial and the requested GTA-style third-person camera represent different input behaviors. I retained both implementations to avoid losing earlier work, while documenting that they require a coordinated interaction design. Another issue was compatibility between the legacy Input API and Unity's newer Input System; selecting Both resolved the test-scene exception. Testing is incomplete until the final island scene and character are available.

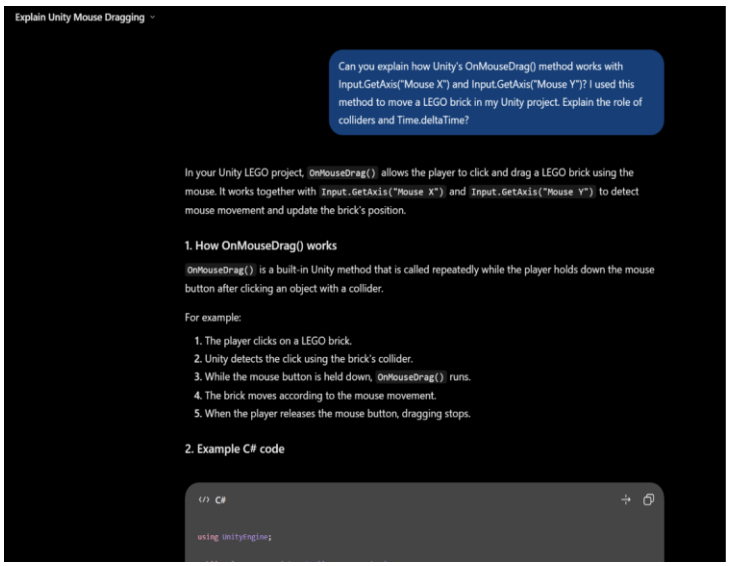
Test / challenge	Observed result	Action / remaining check
Brick and rock dragging	Both objects moved when dragged in the test scene.	Used OnMouseDrag, colliders and reusable prefabs.
Input System exception	Legacy Input.GetAxis initially produced an InvalidOperationException.	Changed Active Input Handling to Both and restarted Unity; dragging worked.
Pink LEGO material	Imported material displayed pink under URP.	Created a URP/Lit material and reassigned it.
Menu layout	Top-right Menu button and full-HD layout adjusted.	Confirm all Start/Resume/Escape interactions in Play mode.
Third-person scripts	Code created and pushed; no final-character test reported.	Attach scripts, assign references and test WASD, camera, terrain and cursor.
Scene integration	Island and character managed by teammates.	Confirm ground colliders, camera selection, merge and animation.

6. References and asset credits

OpenAI (2026) *ChatGPT*. Available at: https://chatgpt.com/ (Accessed: 9 November 2026).
Unity Technologies (n.d.) *Unity Scripting API: Input.GetAxis*. Available at: https://docs.unity3d.com/ScriptReference/Input.GetAxis.html (Accessed: 9 November 2026).
Unity Technologies (n.d.) *Unity Scripting API: CharacterController*. Available at: https://docs.unity3d.com/ScriptReference/CharacterController.html (Accessed: 9 November 2026).

Dmytro Plekhotkin (2023) *Mouse doesn't work with Unity Input System. How to fix: MouseControl*. Available at: https://www.youtube.com/watch?v=6VowIn_gbSc (Accessed: 9 November 2026).





7. Appendix A — Evidence and final check

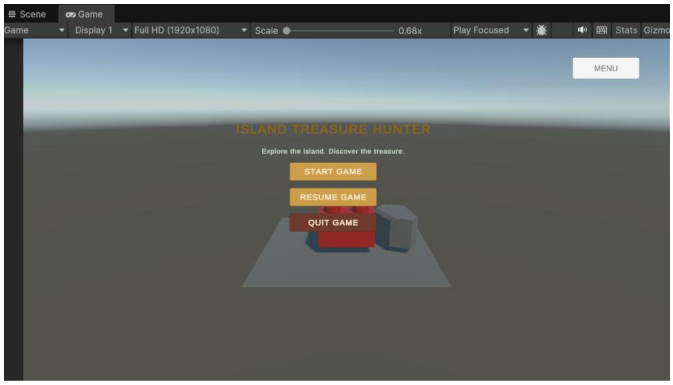


Figure 1:AI Transcript_1

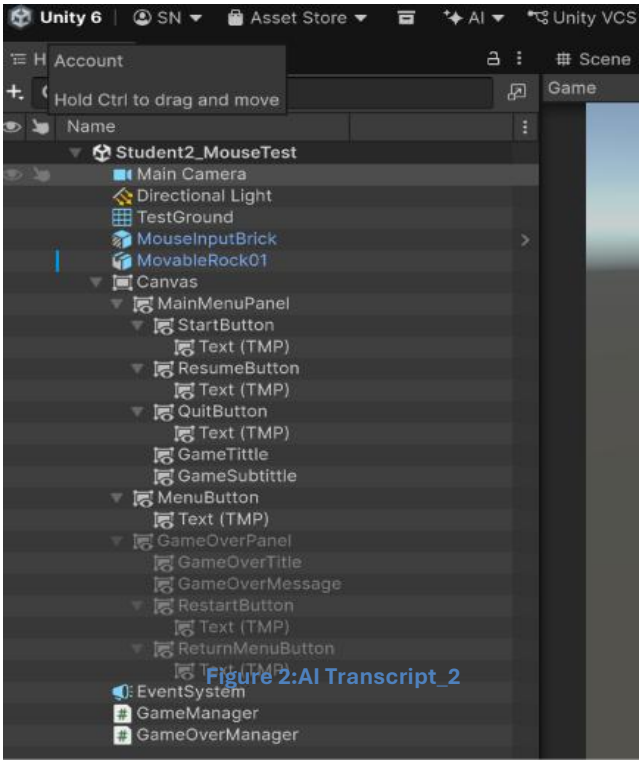


Figure 2:AI Transcript_2

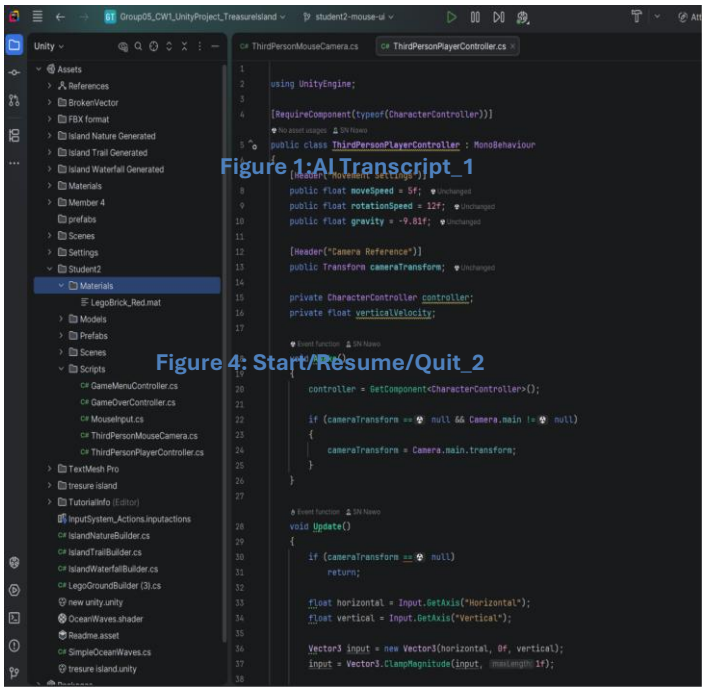


Figure 4: Start/Resume/Quit_2

Figure 3: Start/Resume/Quit

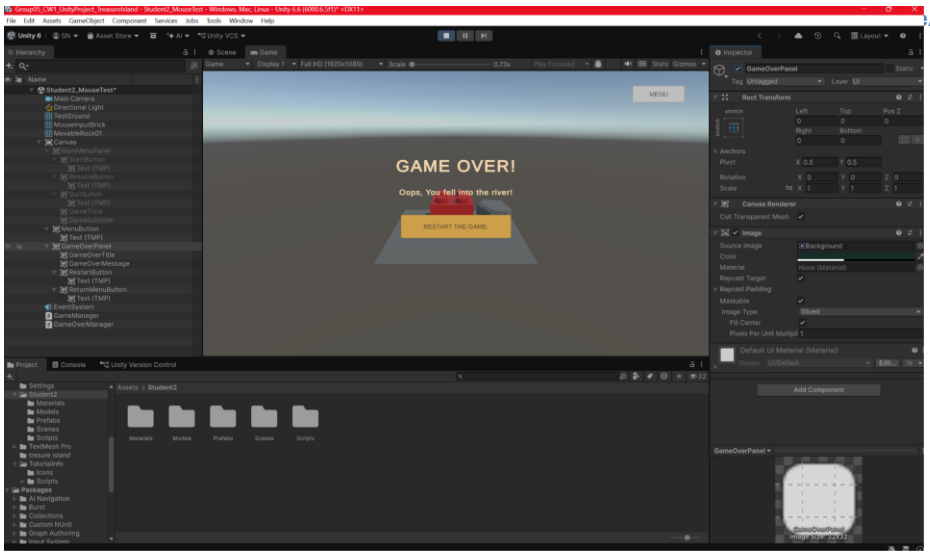


Figure 5: GameOver / Rstart